

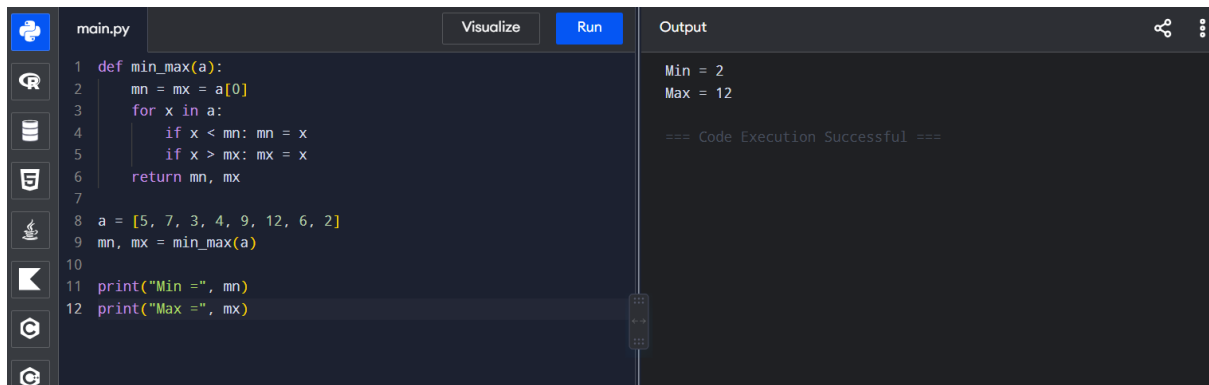
DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

EXPERIMENT :01

Aim

To write a Python program to find the **maximum and minimum values** in an array and display the results for the given test cases.

Code



```
main.py Visualize Run Output
1 def min_max(a):
2     mn = mx = a[0]
3     for x in a:
4         if x < mn: mn = x
5         if x > mx: mx = x
6     return mn, mx
7
8 a = [5, 7, 3, 4, 9, 12, 6, 2]
9 mn, mx = min_max(a)
10
11 print("Min =", mn)
12 print("Max =", mx)
```

Min = 2
Max = 12
=== Code Execution Successful ===

Time Complexity

The program traverses the array only once.

- **Time Complexity: $O(N)$**
- Each element is compared with the current minimum and maximum.

Space Complexity

Only two extra variables, `min_val` and `max_val`, are used.

- **Space Complexity: $O(1)$**

Result

Thus, the Python program successfully finds the **minimum and maximum values** in the given arrays.

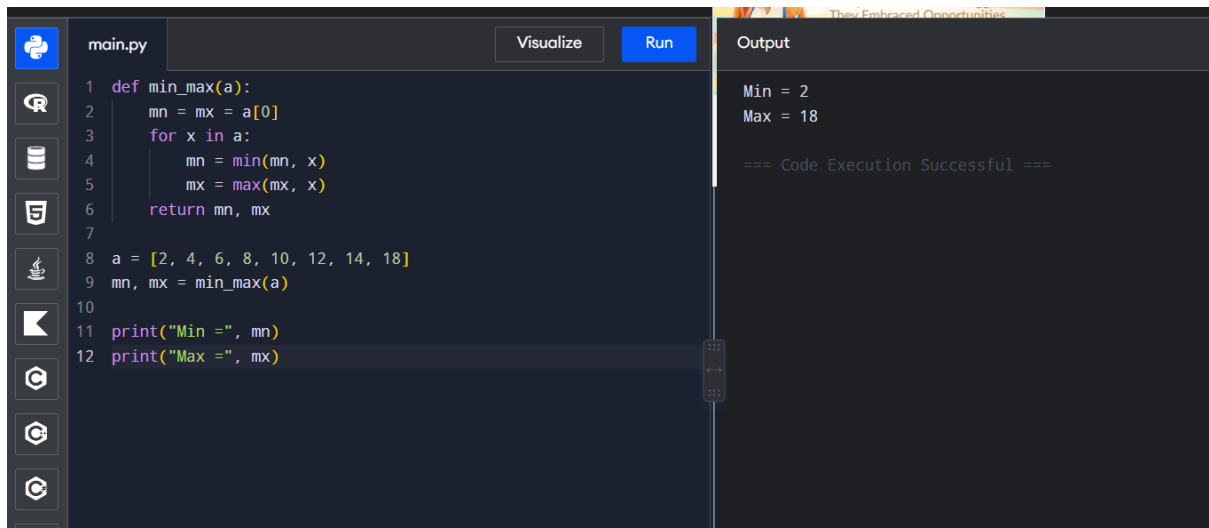
- **Test Case 1:** Min = 2, Max = 12
- **Test Case 2:** Min = 1, Max = 17
- **Test Case 3:** Min = 12, Max = 67

EXPERIMENT :02

Aim

To write a Python program to find the minimum and maximum values in an array.

Code

A screenshot of a Python IDE interface. The main editor window shows a file named 'main.py' with the following code:

```
1 def min_max(a):  
2     mn = mx = a[0]  
3     for x in a:  
4         mn = min(mn, x)  
5         mx = max(mx, x)  
6     return mn, mx  
7  
8 a = [2, 4, 6, 8, 10, 12, 14, 18]  
9 mn, mx = min_max(a)  
10  
11 print("Min =", mn)  
12 print("Max =", mx)
```

On the right side, there is an 'Output' panel showing the results of the program execution:

```
Min = 2  
Max = 18  
  
=== Code Execution Successful ===
```

The IDE also features a left sidebar with various icons for file management and a top bar with 'Visualize' and 'Run' buttons.

Time Complexity

$O(N)$

Space Complexity

$O(1)$

Result

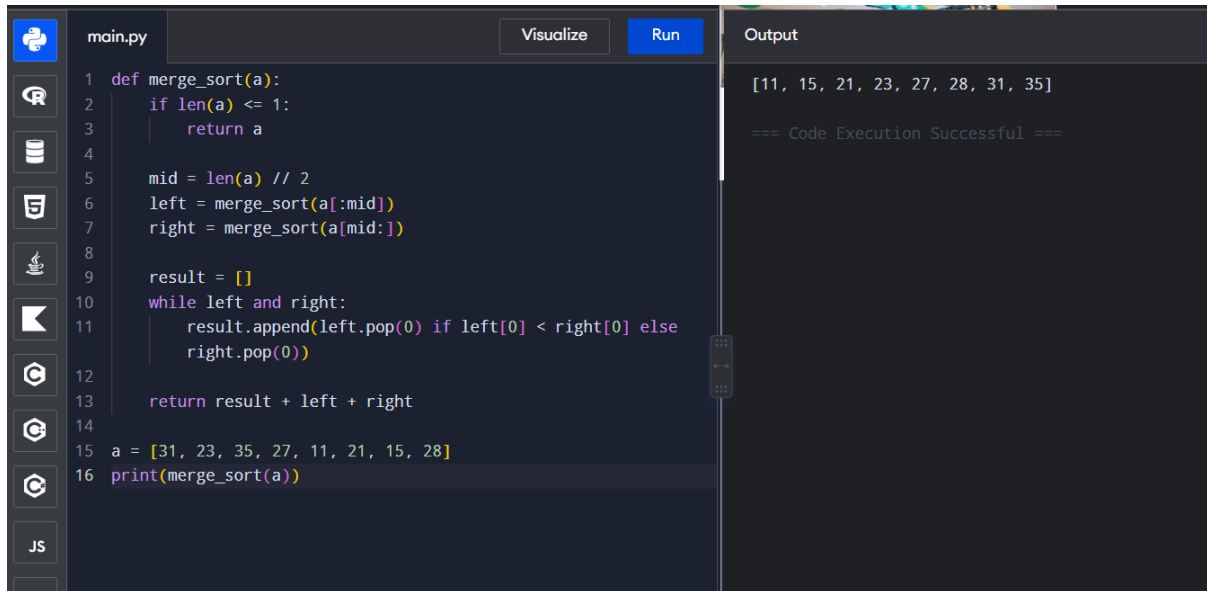
Thus, the program successfully finds the minimum and maximum values in the given array.

EXPERIMENT :03

Aim

To write a Python program to **sort an unsorted array in ascending order using Merge Sort**.

Code



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a Merge Sort algorithm. It defines a function 'merge_sort(a)' that recursively splits the array into halves until it reaches a base case of length 1 or less, then merges the sorted halves back together. The initial array 'a' is [31, 23, 35, 27, 11, 21, 15, 28]. The output of the program is the sorted array [11, 15, 21, 23, 27, 28, 31, 35].

```
1 def merge_sort(a):
2     if len(a) <= 1:
3         return a
4
5     mid = len(a) // 2
6     left = merge_sort(a[:mid])
7     right = merge_sort(a[mid:])
8
9     result = []
10    while left and right:
11        result.append(left.pop(0) if left[0] < right[0] else
12                        right.pop(0))
13
14    return result + left + right
15
16 a = [31, 23, 35, 27, 11, 21, 15, 28]
17 print(merge_sort(a))
```

Output

```
[11, 15, 21, 23, 27, 28, 31, 35]
=== Code Execution Successful ===
```

Time Complexity

$O(N \log N)$

Space Complexity

$O(N)$

Result

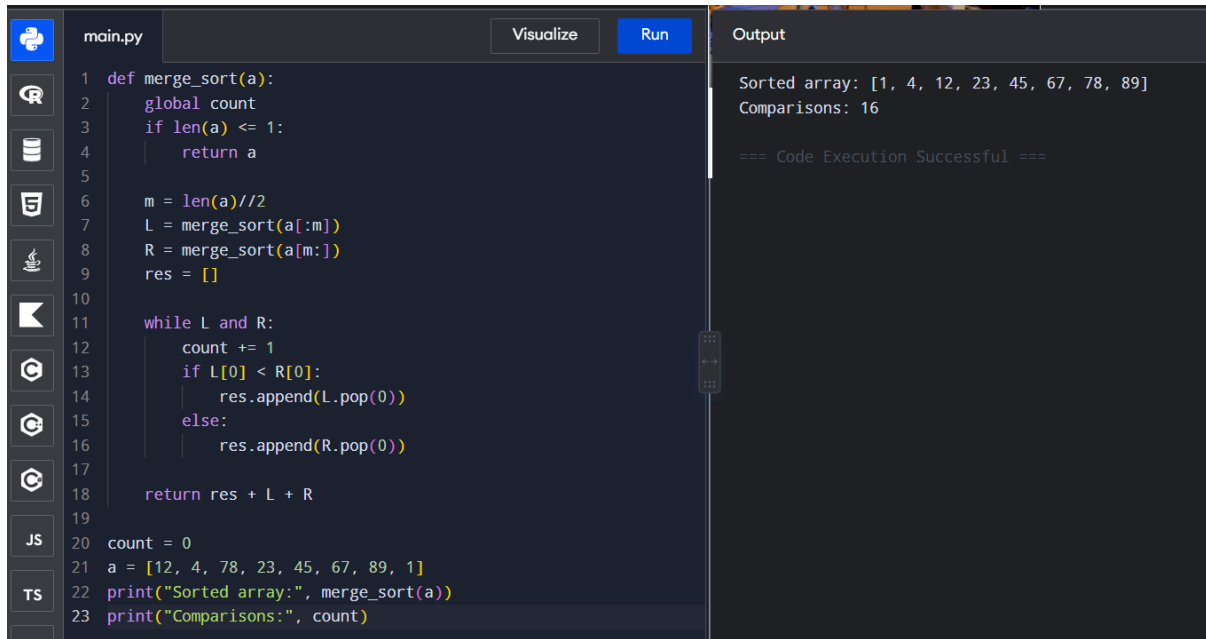
Thus, the given arrays are successfully sorted in **ascending order using Merge Sort**.

EXPERIMENT :04

Aim

To implement **Merge Sort** and count the number of **comparisons** made during the sorting process.

Code



The screenshot shows a Python IDE with a file named 'main.py'. The code implements a Merge Sort algorithm and counts the number of comparisons. The output panel shows the sorted array and the total comparisons.

```
1 def merge_sort(a):
2     global count
3     if len(a) <= 1:
4         return a
5
6     m = len(a)//2
7     L = merge_sort(a[:m])
8     R = merge_sort(a[m:])
9     res = []
10
11     while L and R:
12         count += 1
13         if L[0] < R[0]:
14             res.append(L.pop(0))
15         else:
16             res.append(R.pop(0))
17
18     return res + L + R
19
20 count = 0
21 a = [12, 4, 78, 23, 45, 67, 89, 1]
22 print("Sorted array:", merge_sort(a))
23 print("Comparisons:", count)
```

Output:

```
Sorted array: [1, 4, 12, 23, 45, 67, 78, 89]
Comparisons: 16

=== Code Execution Successful ===
```

Time Complexity

$O(N \log N)$

Space Complexity

$O(N)$

Result

Thus, the Merge Sort algorithm successfully sorts the array in ascending order and counts the number of comparisons performed.

EXPERIMENT :05

Aim

To implement **Quick Sort** using the **first element as the pivot**, display the array after each partition, and recursively sort the sub-arrays.

Code

```
main.py Visualize Run
1 def quick_sort(a, low, high):
2     if low < high:
3         p = partition(a, low, high)
4         print(a)
5         quick_sort(a, low, p - 1)
6         quick_sort(a, p + 1, high)
7
8 def partition(a, low, high):
9     pivot = a[low]
10    i, j = low + 1, high
11
12    while True:
13        while i <= high and a[i] <= pivot:
14            i += 1
15        while a[j] > pivot:
16            j -= 1
17        if i >= j:
18            break
19        a[i], a[j] = a[j], a[i]
20
21    a[low], a[j] = a[j], a[low]
22    return j
23
24 a = [10, 16, 8, 12, 15, 6, 3, 9, 5]
25 quick_sort(a, 0, len(a)-1)
26 print("Sorted:", a)
```

Output

```
[6, 5, 8, 9, 3, 10, 15, 12, 16]
[3, 5, 6, 9, 8, 10, 15, 12, 16]
[3, 5, 6, 9, 8, 10, 15, 12, 16]
[3, 5, 6, 8, 9, 10, 15, 12, 16]
[3, 5, 6, 8, 9, 10, 12, 15, 16]
Sorted: [3, 5, 6, 8, 9, 10, 12, 15, 16]

=== Code Execution Successful ===
```

Time Complexity

- **Best/Average:** $O(N \log N)$
- **Worst:** $O(N^2)$

Space Complexity

$O(N)$ in the worst case due to recursion.

Result

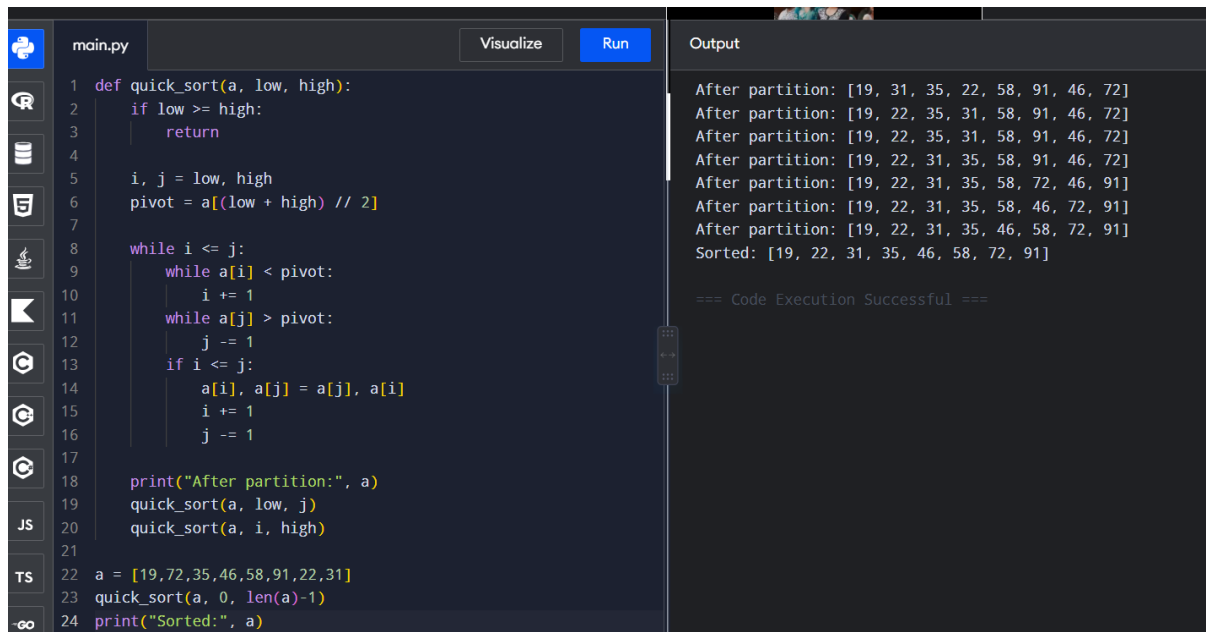
Thus, the **Quick Sort algorithm** successfully sorts the given arrays in ascending order using the **first element as the pivot**.

EXPERIMENT :06

Aim

To implement **Quick Sort** using the **middle element as the pivot**, display the array after each partition, and recursively sort the sub-arrays.

Code



The screenshot shows a code editor with a file named 'main.py'. The code implements a Quick Sort algorithm using the middle element as the pivot. The array is [19, 72, 35, 46, 58, 91, 22, 31]. The output shows the array after each partition and the final sorted array.

```
1 def quick_sort(a, low, high):
2     if low >= high:
3         return
4
5     i, j = low, high
6     pivot = a[(low + high) // 2]
7
8     while i <= j:
9         while a[i] < pivot:
10             i += 1
11         while a[j] > pivot:
12             j -= 1
13         if i <= j:
14             a[i], a[j] = a[j], a[i]
15             i += 1
16             j -= 1
17
18     print("After partition:", a)
19     quick_sort(a, low, j)
20     quick_sort(a, i, high)
21
22 a = [19, 72, 35, 46, 58, 91, 22, 31]
23 quick_sort(a, 0, len(a)-1)
24 print("Sorted:", a)
```

Output:

```
After partition: [19, 31, 35, 22, 58, 91, 46, 72]
After partition: [19, 22, 35, 31, 58, 91, 46, 72]
After partition: [19, 22, 35, 31, 58, 91, 46, 72]
After partition: [19, 22, 31, 35, 58, 91, 46, 72]
After partition: [19, 22, 31, 35, 58, 72, 46, 91]
After partition: [19, 22, 31, 35, 58, 46, 72, 91]
After partition: [19, 22, 31, 35, 46, 58, 72, 91]
Sorted: [19, 22, 31, 35, 46, 58, 72, 91]
```

=== Code Execution Successful ===

Time Complexity

- **Best/Average:** $O(N \log N)$
- **Worst:** $O(N^2)$

Space Complexity

$O(\log N)$ on average due to recursion.

Result

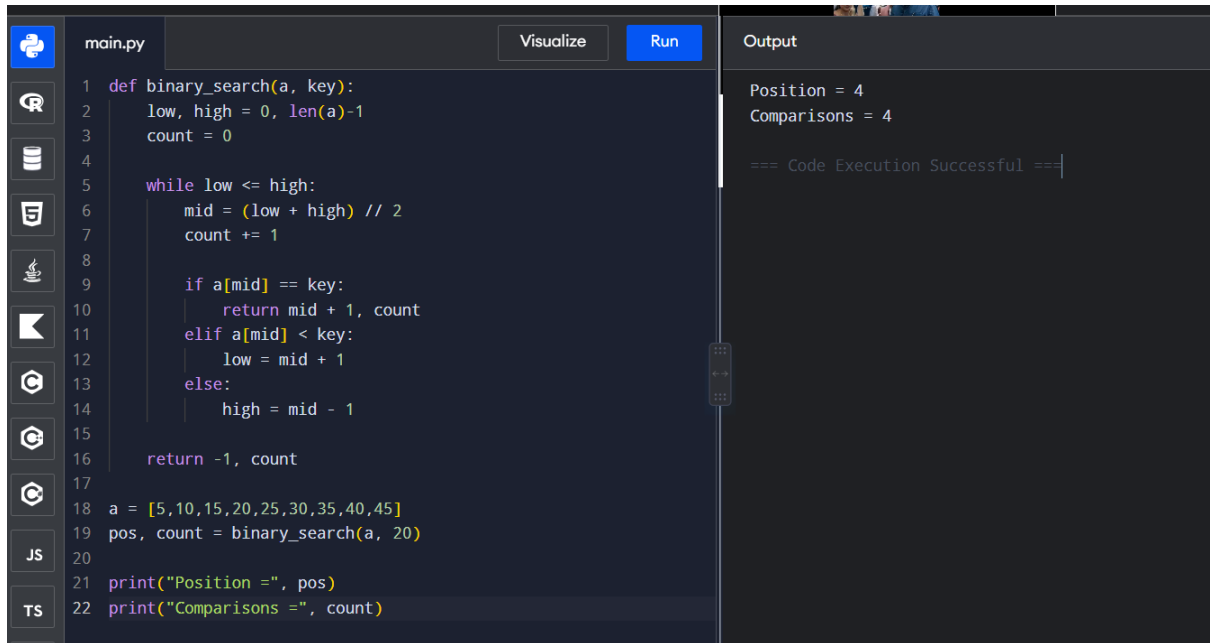
Thus, the **Quick Sort algorithm using the middle element as pivot** successfully sorts the given arrays in ascending order.

EXPERIMENT :07

Aim

To implement **Binary Search** on a sorted array, find the position of the given search key, and count the number of comparisons made.

Code



```
main.py Visualize Run
1 def binary_search(a, key):
2     low, high = 0, len(a)-1
3     count = 0
4
5     while low <= high:
6         mid = (low + high) // 2
7         count += 1
8
9         if a[mid] == key:
10            return mid + 1, count
11        elif a[mid] < key:
12            low = mid + 1
13        else:
14            high = mid - 1
15
16    return -1, count
17
18 a = [5,10,15,20,25,30,35,40,45]
19 pos, count = binary_search(a, 20)
20
21 print("Position =", pos)
22 print("Comparisons =", count)
```

Output

```
Position = 4
Comparisons = 4

=== Code Execution Successful ===
```

Time Complexity

- **Best Case:** $O(1)$
- **Average/Worst Case:** $O(\log N)$

Space Complexity

$O(1)$

Result

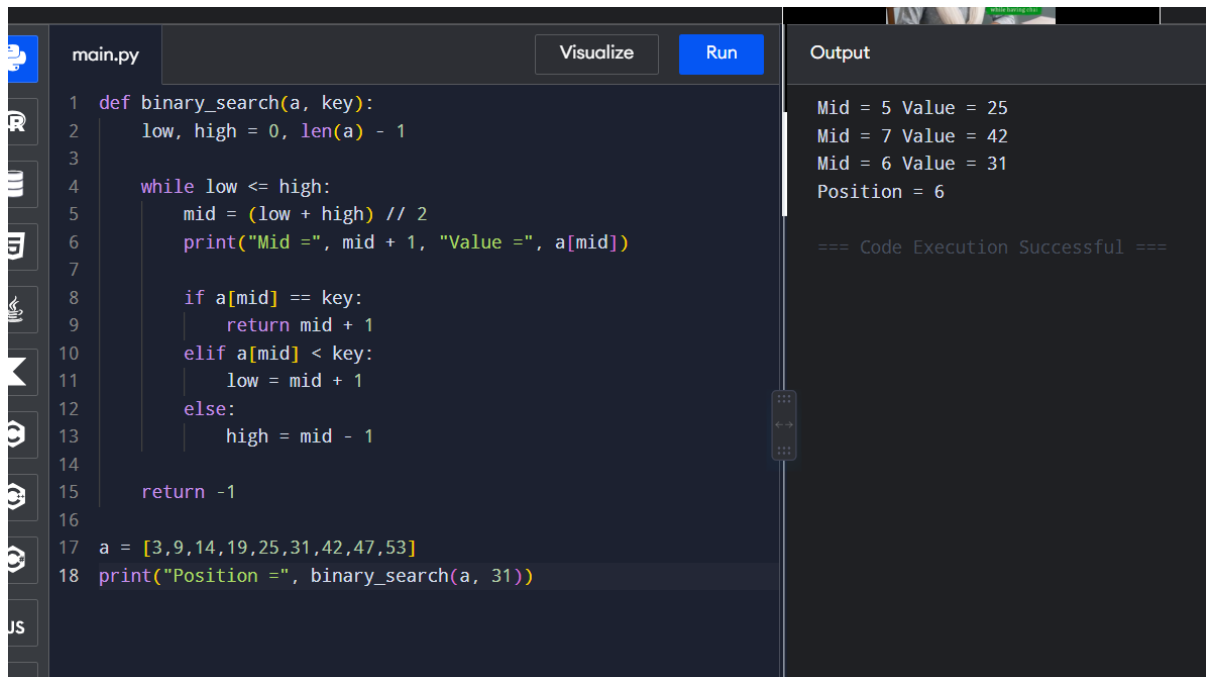
Thus, the Binary Search algorithm successfully finds the required element and counts the comparisons made during the search.

EXPERIMENT :08

Aim

To implement **Binary Search** on a sorted array, show the mid-point calculations and search steps, and explain the effect of using an unsorted array.

Code



The screenshot shows a code editor with a file named 'main.py'. The code implements a binary search function. The array 'a' is [3, 9, 14, 19, 25, 31, 42, 47, 53]. The search key is 31. The output shows the mid-point calculations: Mid = 5 Value = 25, Mid = 7 Value = 42, Mid = 6 Value = 31, and Position = 6. The code execution is successful.

```
1 def binary_search(a, key):
2     low, high = 0, len(a) - 1
3
4     while low <= high:
5         mid = (low + high) // 2
6         print("Mid =", mid + 1, "Value =", a[mid])
7
8         if a[mid] == key:
9             return mid + 1
10        elif a[mid] < key:
11            low = mid + 1
12        else:
13            high = mid - 1
14
15    return -1
16
17 a = [3,9,14,19,25,31,42,47,53]
18 print("Position =", binary_search(a, 31))
```

Output

```
Mid = 5 Value = 25
Mid = 7 Value = 42
Mid = 6 Value = 31
Position = 6

=== Code Execution Successful ===
```

Complexity

- Binary Search Time: $O(\log N)$
- Binary Search Space: $O(1)$
- If sorting is required first: $O(N \log N)$

Result

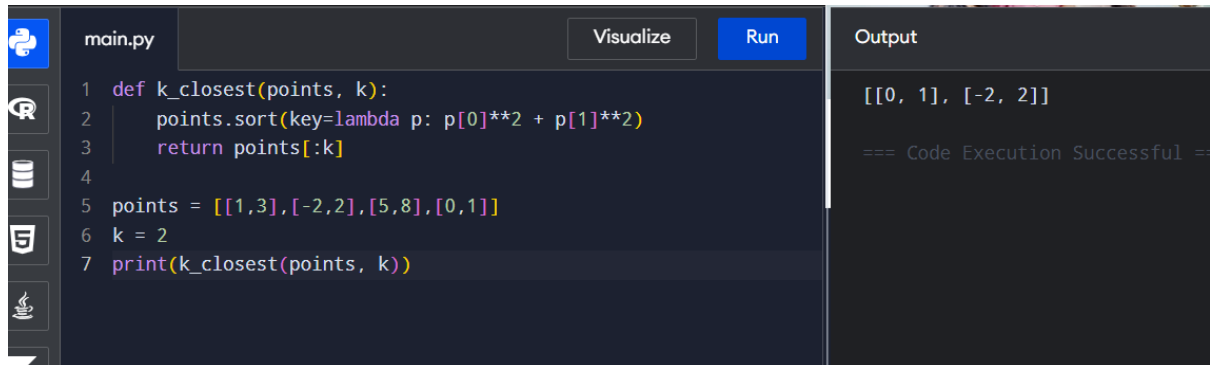
Thus, Binary Search successfully finds **31 at position 6** in the sorted array. It is efficient with $O(\log N)$ time, but it is **not reliable on an unsorted array**.

EXPERIMENT :09

Aim

To write a Python program to find the **K closest points to the origin (0,0)** using the distance formula.

Code

A screenshot of a Python IDE interface. The main editor window, titled 'main.py', contains the following Python code:

```
1 def k_closest(points, k):  
2     points.sort(key=lambda p: p[0]**2 + p[1]**2)  
3     return points[:k]  
4  
5 points = [[1,3],[-2,2],[5,8],[0,1]]  
6 k = 2  
7 print(k_closest(points, k))
```

At the top of the editor, there are buttons for 'Visualize' and 'Run'. On the left side, there is a vertical toolbar with icons for Python, Jupyter Notebook, and other tools. To the right of the editor, there is an 'Output' panel showing the result of the execution:

```
[[0, 1], [-2, 2]]  
  
=== Code Execution Successful ===
```

Time Complexity

$O(N \log N)$ — due to sorting the points.

Space Complexity

$O(N)$ — for the sorting operation.

Result

Thus, the program successfully finds the **K closest points to the origin (0,0)** for all the given test cases.

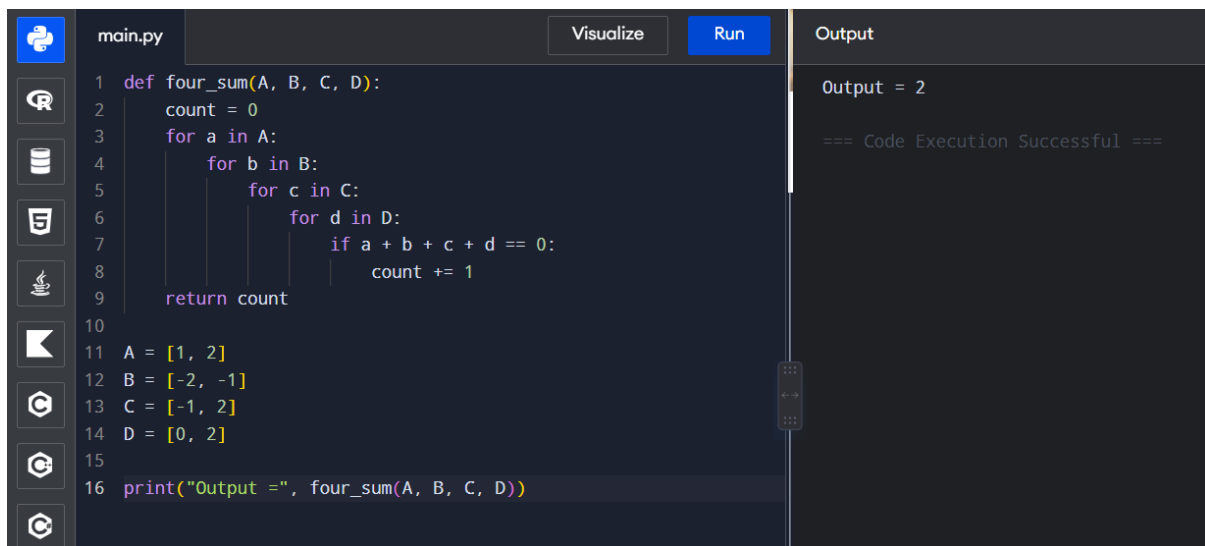
EXPERIMENT :10

Aim

To write a Python program to count the number of tuples (i, j, k, l) such that:

$$A[i] + B[j] + C[k] + D[l] = 0$$

Code



```
main.py Visualize Run Output
1 def four_sum(A, B, C, D):
2     count = 0
3     for a in A:
4         for b in B:
5             for c in C:
6                 for d in D:
7                     if a + b + c + d == 0:
8                         count += 1
9     return count
10
11 A = [1, 2]
12 B = [-2, -1]
13 C = [-1, 2]
14 D = [0, 2]
15
16 print("Output =", four_sum(A, B, C, D))

Output = 2
=== Code Execution Successful ===
```

Time Complexity

$O(N^4)$

Space Complexity

$O(1)$

Result

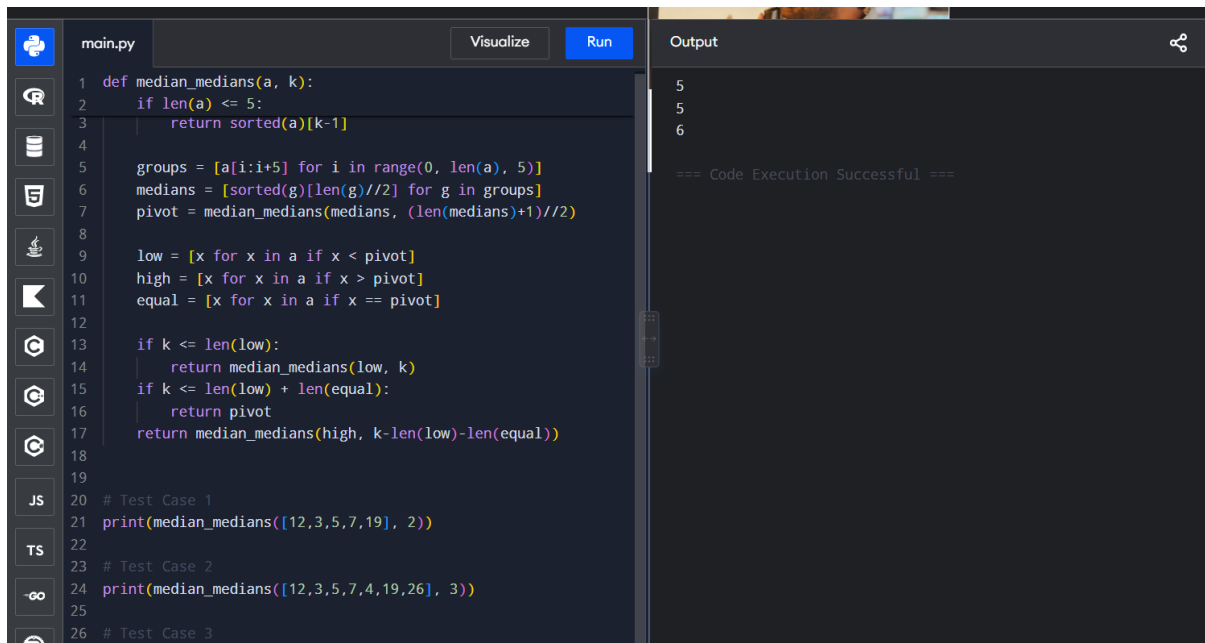
Thus, the program successfully counts the number of tuples whose sum is **zero**.

EXPERIMENT :11

Aim

To implement the **Median of Medians algorithm** to find the **k-th smallest element** in an unsorted array with guaranteed **$O(N)$** worst-case time complexity.

Code



```
1 def median_medians(a, k):
2     if len(a) <= 5:
3         return sorted(a)[k-1]
4
5     groups = [a[i:i+5] for i in range(0, len(a), 5)]
6     medians = [sorted(g)[len(g)//2] for g in groups]
7     pivot = median_medians(medians, (len(medians)+1)//2)
8
9     low = [x for x in a if x < pivot]
10    high = [x for x in a if x > pivot]
11    equal = [x for x in a if x == pivot]
12
13    if k <= len(low):
14        return median_medians(low, k)
15    if k <= len(low) + len(equal):
16        return pivot
17    return median_medians(high, k-len(low)-len(equal))
18
19
20 # Test Case 1
21 print(median_medians([12,3,5,7,19], 2))
22
23 # Test Case 2
24 print(median_medians([12,3,5,7,4,19,26], 3))
25
26 # Test Case 3
```

Output

```
5
5
6

=== Code Execution Successful ===
```

Time Complexity

$O(N)$ worst-case.

Space Complexity

$O(N)$ due to the temporary groups and partitions.

Result

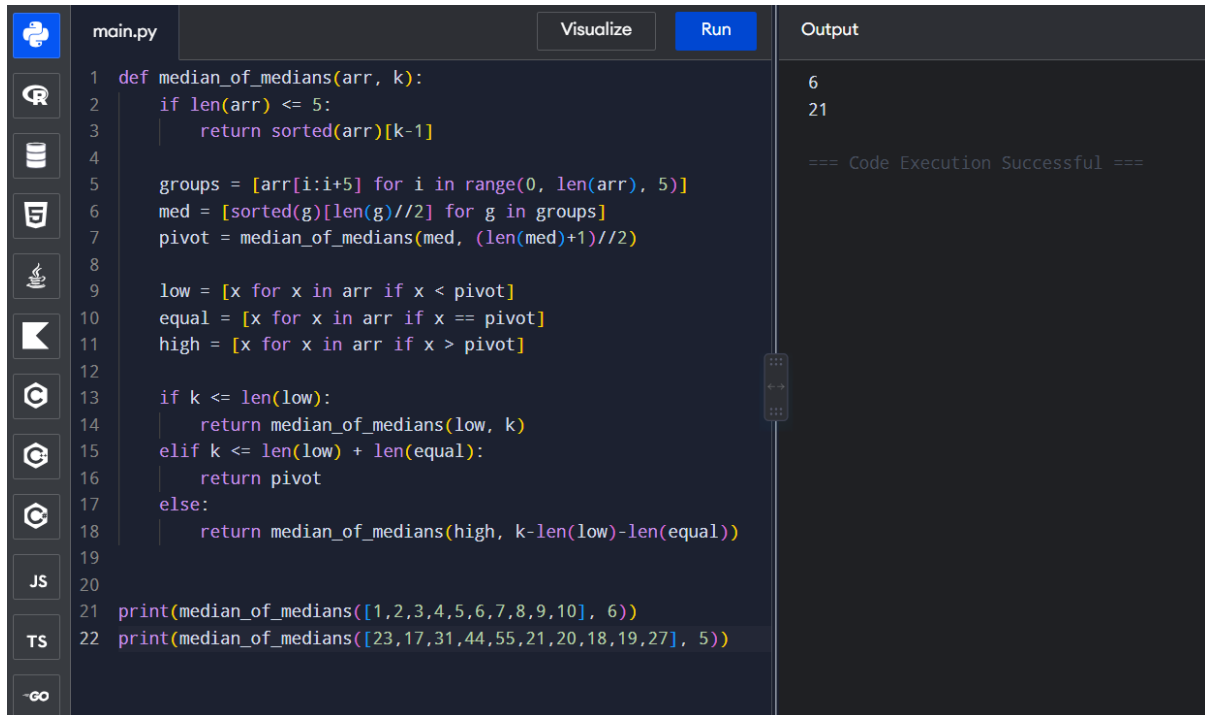
Thus, the **Median of Medians algorithm** successfully finds the **k-th smallest element** in all the given test cases while providing a guaranteed **linear worst-case time complexity**.

EXPERIMENT :12

Aim

To implement the **Median of Medians algorithm** to find the **k-th smallest element** in an unsorted array.

Code



```
1 def median_of_medians(arr, k):
2     if len(arr) <= 5:
3         return sorted(arr)[k-1]
4
5     groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
6     med = [sorted(g)[len(g)//2] for g in groups]
7     pivot = median_of_medians(med, (len(med)+1)//2)
8
9     low = [x for x in arr if x < pivot]
10    equal = [x for x in arr if x == pivot]
11    high = [x for x in arr if x > pivot]
12
13    if k <= len(low):
14        return median_of_medians(low, k)
15    elif k <= len(low) + len(equal):
16        return pivot
17    else:
18        return median_of_medians(high, k-len(low)-len(equal))
19
20
21 print(median_of_medians([1,2,3,4,5,6,7,8,9,10], 6))
22 print(median_of_medians([23,17,31,44,55,21,20,18,19,27], 5))
```

Output

```
6
21

=== Code Execution Successful ===
```

Time Complexity

$O(N)$ — worst-case.

Space Complexity

$O(N)$

Result

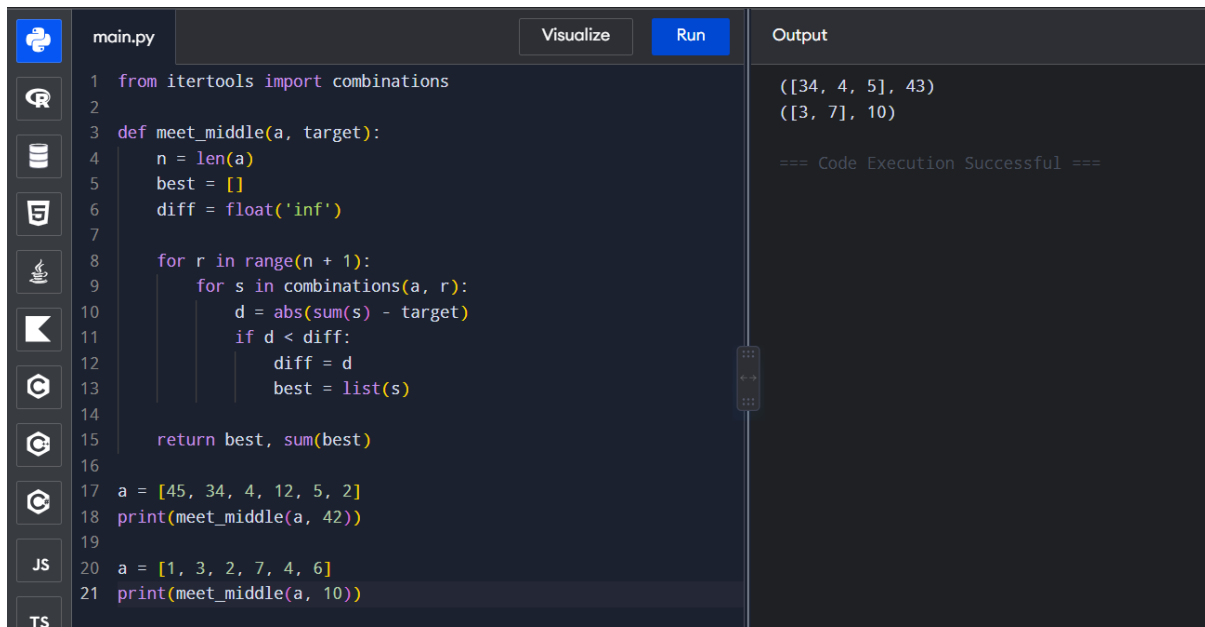
Thus, the `median_of_medians(arr, k)` function successfully finds the **k-th smallest element** in the given unsorted arrays.

EXPERIMENT :13

Aim

To implement the **Meet in the Middle technique** to find the subset whose sum is closest to the given target sum.

Code



```
main.py Visualize Run Output
1 from itertools import combinations
2
3 def meet_middle(a, target):
4     n = len(a)
5     best = []
6     diff = float('inf')
7
8     for r in range(n + 1):
9         for s in combinations(a, r):
10             d = abs(sum(s) - target)
11             if d < diff:
12                 diff = d
13                 best = list(s)
14
15     return best, sum(best)
16
17 a = [45, 34, 4, 12, 5, 2]
18 print(meet_middle(a, 42))
19
20 a = [1, 3, 2, 7, 4, 6]
21 print(meet_middle(a, 10))
```

Output

```
([34, 4, 5], 43)
([3, 7], 10)

=== Code Execution Successful ===
```

Time Complexity

$O(2^{(N/2)} \times N)$ approximately.

Space Complexity

$O(2^{(N/2)})$

Result

Thus, the program successfully finds the subset whose sum is **closest to the target** using the Meet in the Middle approach.

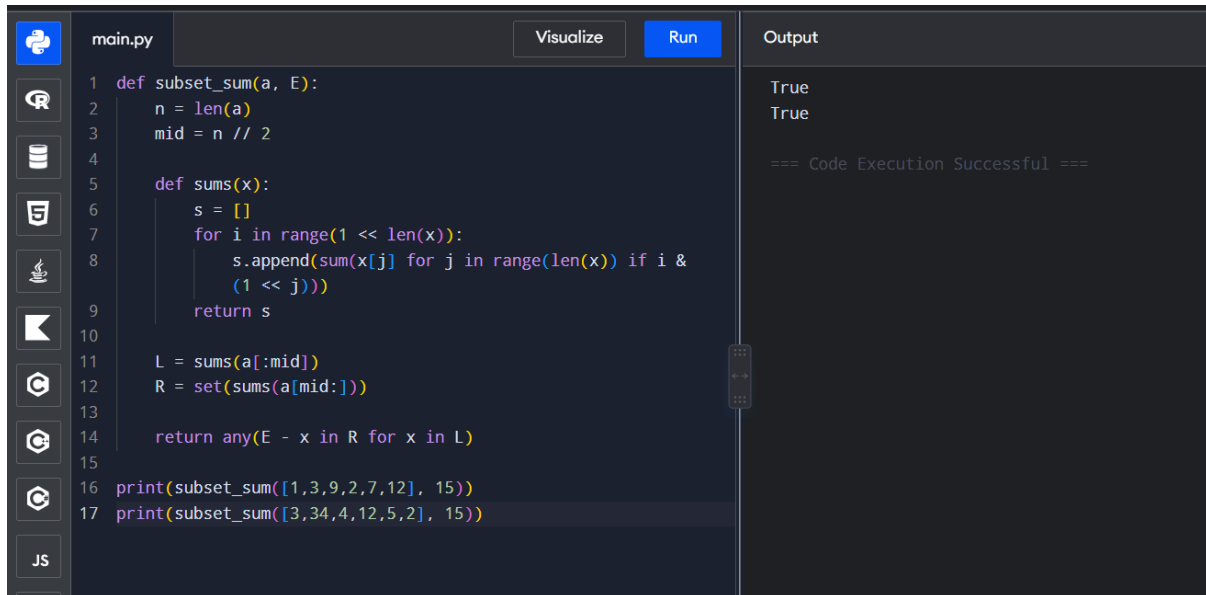
- **(a)** Closest subset = {34, 4, 2}, Sum = **40**
- **(b)** Closest subset = {3, 7}, Sum = **10**

EXPERIMENT :14

Aim

To implement the **Meet in the Middle technique** to determine whether any subset of the given array has a sum exactly equal to the target E.

Code



```
main.py Visualize Run Output
1 def subset_sum(a, E):
2     n = len(a)
3     mid = n // 2
4
5     def sums(x):
6         s = []
7         for i in range(1 << len(x)):
8             s.append(sum(x[j] for j in range(len(x)) if i &
9                         (1 << j)))
10        return s
11
12    L = sums(a[:mid])
13    R = set(sums(a[mid:]))
14
15    return any(E - x in R for x in L)
16
17 print(subset_sum([1,3,9,2,7,12], 15))
18 print(subset_sum([3,34,4,12,5,2], 15))
```

True
True
=== Code Execution Successful ===

Time Complexity

$O(2^{(N/2)})$

Space Complexity

$O(2^{(N/2)})$

Result

Thus, the **Meet in the Middle algorithm** successfully determines whether an exact subset sum exists.

- Test Case (a): True
- Test Case (b): True

EXPERIMENT :15

15. Strassen's Matrix Multiplication

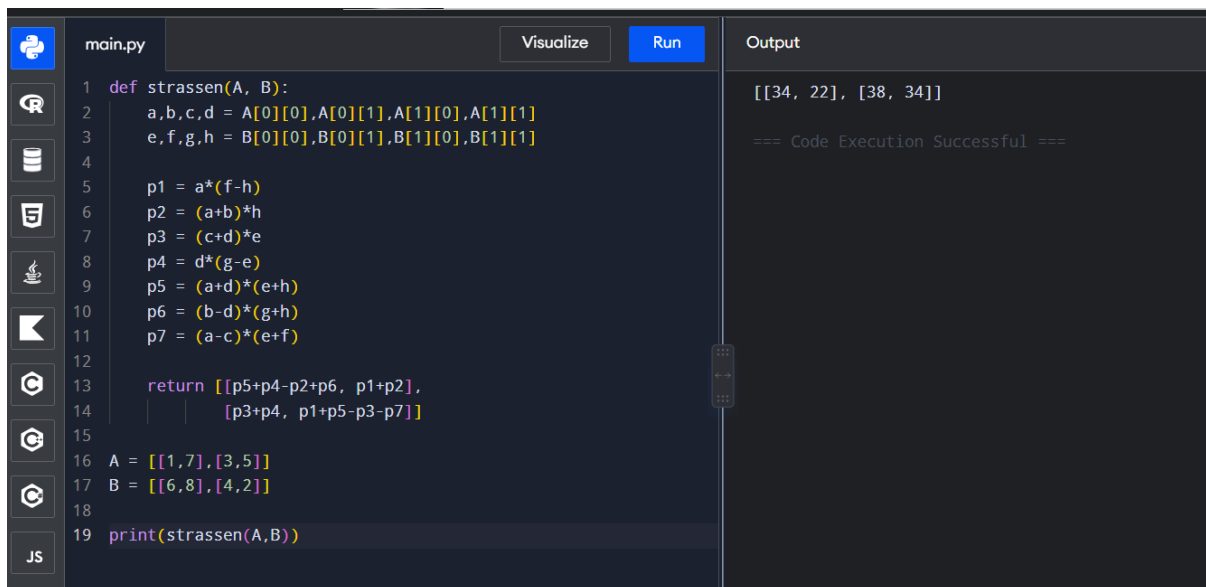
Aim

To implement **Strassen's Matrix Multiplication algorithm** for multiplying two 2x2 matrices.

Input

$$A = \begin{bmatrix} 1 & 7 \\ 3 & 5 \end{bmatrix}, B = \begin{bmatrix} 6 & 8 \\ 4 & 2 \end{bmatrix}$$

Code



```
main.py Visualize Run Output
1 def strassen(A, B):
2     a,b,c,d = A[0][0],A[0][1],A[1][0],A[1][1]
3     e,f,g,h = B[0][0],B[0][1],B[1][0],B[1][1]
4
5     p1 = a*(f-h)
6     p2 = (a+b)*h
7     p3 = (c+d)*e
8     p4 = d*(g-e)
9     p5 = (a+d)*(e+h)
10    p6 = (b-d)*(g+h)
11    p7 = (a-c)*(e+f)
12
13    return [[p5+p4-p2+p6, p1+p2],
14            [p3+p4, p1+p5-p3-p7]]
15
16 A = [[1,7],[3,5]]
17 B = [[6,8],[4,2]]
18
19 print(strassen(A,B))

[[34, 22], [38, 34]]
=== Code Execution Successful ===
```

Time Complexity

$O(n^{2.81})$ using Strassen's algorithm.

Space Complexity

$O(n^2)$

Result

Thus, Strassen's matrix multiplication algorithm successfully computes the product matrix:

$$C = \begin{bmatrix} 34 & 22 \\ 38 & 34 \end{bmatrix}$$

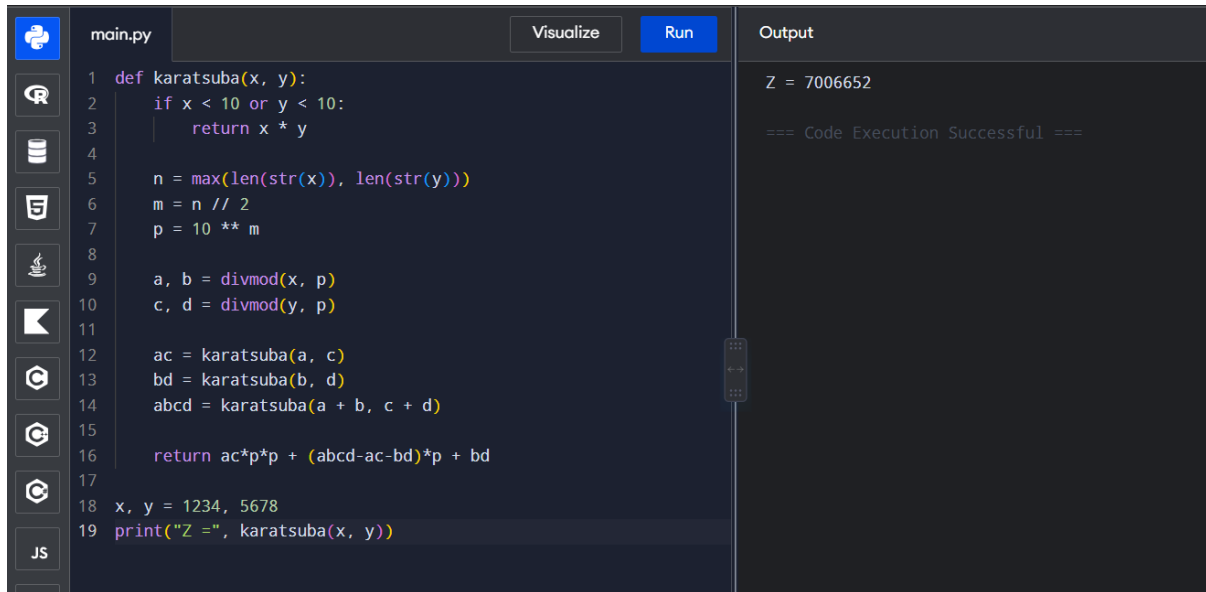
EXPERIMENT :16

16. Karatsuba Multiplication

Aim

To implement the **Karatsuba multiplication algorithm** to compute the product of two integers.

Code



```
main.py Visualize Run Output
1 def karatsuba(x, y):
2     if x < 10 or y < 10:
3         return x * y
4
5     n = max(len(str(x)), len(str(y)))
6     m = n // 2
7     p = 10 ** m
8
9     a, b = divmod(x, p)
10    c, d = divmod(y, p)
11
12    ac = karatsuba(a, c)
13    bd = karatsuba(b, d)
14    abcd = karatsuba(a + b, c + d)
15
16    return ac*p*p + (abcd-ac-bd)*p + bd
17
18 x, y = 1234, 5678
19 print("Z =", karatsuba(x, y))
```

Z = 7006652

=== Code Execution Successful ===

Time Complexity

$$O(n^{\log_2 3}) \approx O(n^{1.58})$$

Space Complexity

$O(n)$ due to recursion.

Result

Thus, the **Karatsuba algorithm** successfully computes:

$$1234 \times 5678 = 7006652$$